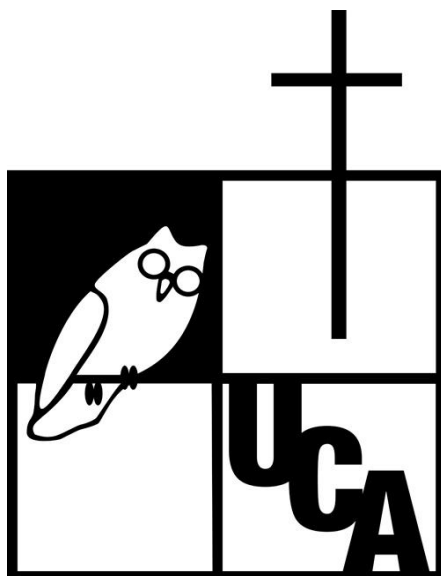


Documento técnico - Sistema de Gestión Clínica Veterinaria



Kevin Daniel Aguilar Cerritos 00026825

Orlando Vladimir Carcamo Espinal 00370825

Walter Giovanni Flores Flores 00111925

Axel Moises Merino Cabrera 00071425

Josué Barahona Tario 00092625

Rodrigo José Villeda Rodezno 00001825

Facultad De Ingeniería Y Arquitectura

Universidad Centroamericana “José Simeon Cañas”

Bases De Datos: Ing. James Humberstone

Índice

1. Descripción del escenario y reglas de negocio	4
1.1. Caso de negocio	4
1.2. Reglas de negocio	5
2. Diagrama Entidad-Relación (notación Chen).....	7
2.1. Entidades y atributos.....	8
2.2. Relaciones, cardinalidad y participación.....	9
3. Esquema relacional normalizado (3FN)	11
3.1. Reglas de transformación aplicadas	11
3.2. Esquema lógico resultante	12
3.3. Mapeo de relaciones a llaves foráneas	14
3.4. Verificación de la Tercera Forma Normal	15
4. Diccionario de datos	16
veterinario	16
especialidad.....	17
veterinario_especialidad.....	17
propietario	18
especie.....	19
mascota	20
cita.....	21
diagnostico	22

tratamiento	23
medicamento	24
detalle_tratamiento.....	25
alergia.....	26
factura	27
5. Documentación de consultas más frecuentes	28
5.1. Historial médico completo de una mascota	28
5.2. Reporte de facturación mensual agregado por método de pago.....	29
5.3. Medicamentos prescritos con mayor frecuencia	30
5.4. Veterinario con mayor número de citas en el trimestre actual.....	30
5.5. Propietarios con mascotas sin citas en los últimos 6 meses.....	31
6. Funciones, procedimientos almacenados y disparadores.....	32
6.1. Función: fn_historial_clinico	32
6.2. Procedimiento almacenado: sp_RegistrarCita	33
6.3. Disparador (trigger): tr_ValidarAlergiasAntesTratamiento.....	35

1. Descripción del escenario y reglas de negocio

1.1. Caso de negocio

El presente documento describe el diseño de una base de datos pensada para una clínica veterinaria. El objetivo principal de este sistema es llevar un control ordenado de toda la información que se maneja dentro de la clínica, desde los propietarios y sus mascotas hasta las citas, los diagnósticos y la facturación. De esta manera se busca centralizar los datos en un solo lugar y evitar que la información se pierda o se duplique al momento de atender a los pacientes.

El funcionamiento del sistema gira en torno a la cita, que es el momento en el que un veterinario atiende a una mascota. Cada mascota pertenece a un propietario y a una especie determinada, y durante la cita el veterinario puede generar uno o más diagnósticos. A partir de estos diagnósticos se definen los tratamientos, los cuales pueden incluir medicamentos con su respectiva duración. Además, el sistema registra las alergias de cada mascota, de modo que se pueda evitar que se le recete un medicamento que pueda hacerle daño. Finalmente, al terminar cada cita se genera una factura con el cobro correspondiente a los servicios brindados

1.2. Reglas de negocio

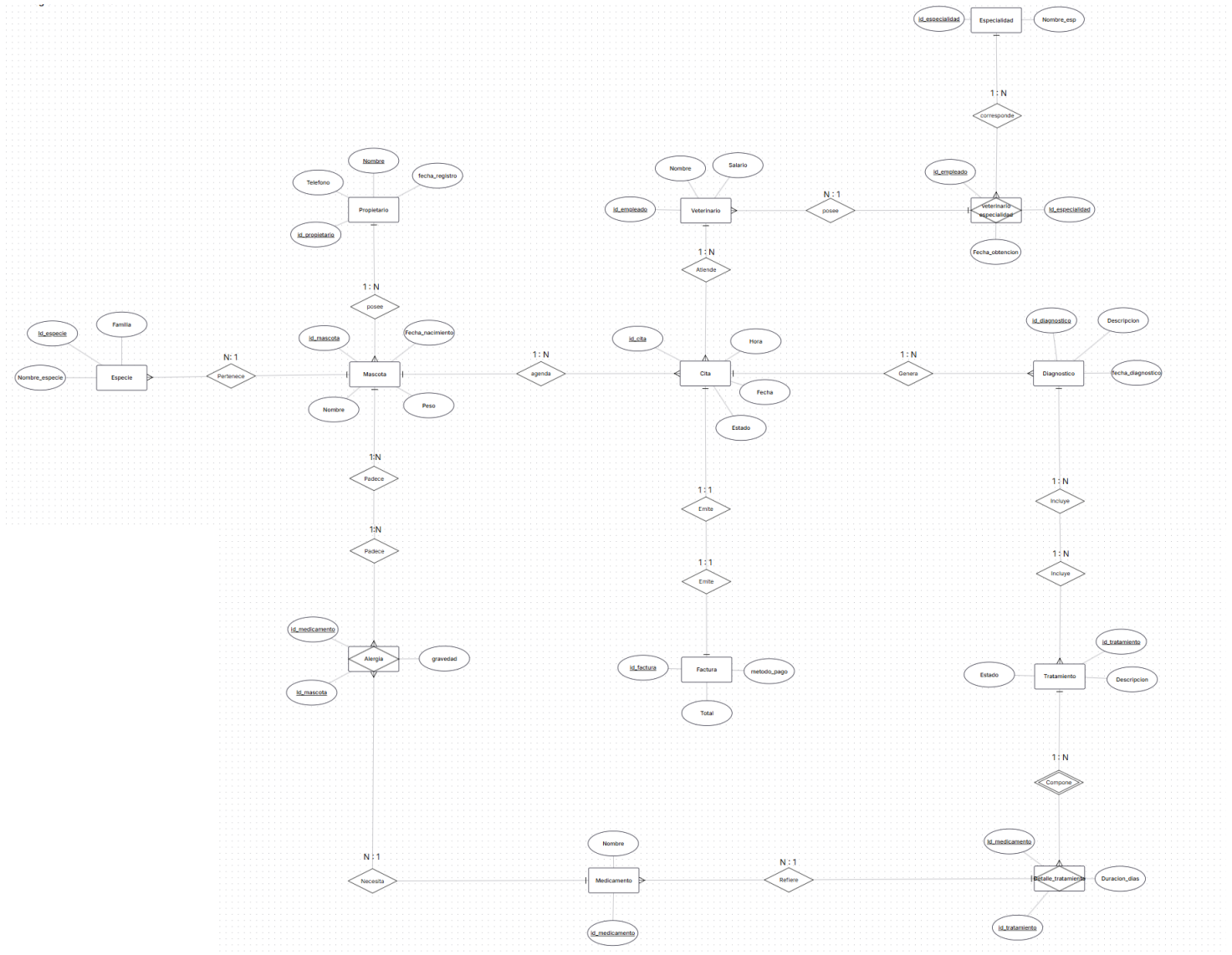
Para entender el porqué de las relaciones que aparecen en el modelo, a continuación, se presentan las reglas de negocio que definen cómo funciona la clínica. Estas reglas son las que justifican las cardinalidades (1:N, N:M y 1:1) que se observan en el diagrama:

1. **RN-01.** Un propietario puede registrar una o varias mascotas; cada mascota pertenece a un único propietario. (*Relación 1:N*)
2. **RN-02.** Cada mascota pertenece a exactamente una especie, mientras que una especie agrupa a muchas mascotas. (*Relación 1:N*)
3. **RN-03.** Una mascota puede agendar múltiples citas a lo largo del tiempo; cada cita corresponde a una sola mascota. (*Relación 1:N*)
4. **RN-04.** Cada cita es atendida por un único veterinario; un veterinario atiende muchas citas. (*Relación 1:N*)
5. **RN-05.** Un veterinario puede poseer varias especialidades y una misma especialidad puede ser ejercida por varios veterinarios. (*Relación N:M, resuelta con la tabla asociativa veterinario_especialidad*)
6. **RN-06.** Cada cita genera uno o más diagnósticos; cada diagnóstico proviene de una sola cita. (*Relación 1:N*)
7. **RN-07.** Un diagnóstico puede derivar en uno o varios tratamientos; cada tratamiento responde a un único diagnóstico. (*Relación 1:N*)
8. **RN-08.** Un tratamiento puede involucrar varios medicamentos, y un medicamento puede formar parte de varios tratamientos. La asociación almacena atributos propios como la duración. (*Relación N:M, resuelta con detalle_tratamiento*)

9. **RN-09.** Una mascota puede presentar alergia a uno o varios medicamentos, y un medicamento puede provocar alergia en varias mascotas. (*Relación N:M, resuelta con alergia*)
10. **RN-10.** Cada cita genera exactamente una factura, y cada factura corresponde a una única cita. (*Relación 1:1*)
11. **RN-11.** No se permite registrar dos citas para el mismo veterinario en idéntica fecha y hora, garantizando la disponibilidad real de la agenda.
12. **RN-12.** No se permite prescribir a una mascota un medicamento al cual presenta una alergia previamente registrada.

2. Diagrama Entidad-Relación (notación Chen)

El modelo conceptual de la base de datos se representó utilizando la notación de Chen. En esta notación las entidades se dibujan como rectángulos, los atributos como óvalos y las relaciones como rombos que conectan a las entidades entre sí. La cardinalidad, es decir, la cantidad de



2.1. Entidades y atributos

Entidad	Tipo	Atributos (identificador en primer lugar)
propietario	Fuerte	id_propietario, nombre_propietario, telefono, fecha_registro
mascota	Fuerte	id_mascota, nombre_mascota, peso, fecha_nacimiento
especie	Fuerte	id_especie, nombre_especie, familia
veterinario	Fuerte	id_empleado, nombre_veterinario, salario
especialidad	Fuerte	id_especialidad, nombre_especialidad
cita	Fuerte	id_cita, fecha, hora, estado
diagnostico	Fuerte	id_diagnostico, fecha_diagnostico, descripcion
tratamiento	Fuerte	id_tratamiento, descripcion, estado

medicamento	Fuerte	id_medicamento, nombre_medicamento
factura	Fuerte	id_factura, total, metodo_pago
veterinario_especialidad	Asociativa	fecha_obtencion (+ id_empleado e id_especialidad)
detalle_tratamiento	Asociativa	duracion_dias (+ id_tratamiento e id_medicamento)
alergia	Asociativa	gravedad (+ id_mascota e id_medicamento)

2.2. Relaciones, cardinalidad y participación

Relación	Entidades	Cardinalidad	Participación total en
posee	propietario – mascota	1:N	mascota (toda mascota tiene dueño)
Pertenece	especie – mascota	1:N	mascota (toda mascota tiene especie)
agenda	mascota – cita	1:N	cita (toda cita es de una mascota)

Atiende	veterinario – cita	1:N	cita (toda cita tiene veterinario)
corresponde / posee	veterinario – especialidad	N:M	veterinario_especialidad
Genera	cita – diagnostico	1:N	diagnostico (proviene de una cita)
Incluye	diagnostico – tratamiento	1:N	tratamiento (responde a un diagnóstico)
Compone / Refiere	tratamiento – medicamento	N:M	detalle_tratamiento
Padece / Necesita	mascota – medicamento	N:M	alergia
Emite	cita – factura	1:1	factura (toda factura nace de una cita)

[illegible]

Una vez definido el modelo conceptual en notación Chen, el siguiente paso fue transformarlo en el esquema relacional, que es la forma en la que realmente se implementan las tablas dentro de la base de datos. Para realizar esta conversión se aplicaron las siguientes reglas:

- 11

4. **Entidad asociativa con atributos propios → tabla con llave compuesta.** Es el caso de `detalle_tratamiento`, que además de unir dos tablas guarda información propia de esa relación, como la duración del medicamento.
5. **Atributos atómicos.** Todos los atributos guardan un solo valor, por lo que el modelo cumple desde un inicio con la Primera Forma Normal.

3.2. Esquema lógico resultante

Convención: **[PK]** = llave primaria; **[FK]** = llave foránea con la tabla referenciada.

veterinario (`id_empleado` [PK], `nombre_veterinario`, `salario`)

especialidad (`id_especialidad` [PK], `nombre_especialidad`)

veterinario_especialidad (`id_especialidad` [PK, FK → especialidad],
`id_empleado` [PK, FK → veterinario],
`fecha_obtencion`)

propietario (`id_propietario` [PK], `nombre_propietario`, `telefono`,
`fecha_registro`)

especie (`id_especie` [PK], `nombre_especie`, `familia`)

mascota (`id_mascota` [PK], `nombre_mascota`, `peso`, `fecha_nacimiento`,
`id_especie` [FK → especie],
`id_propietario` [FK → propietario])

cita (id_cita [PK], fecha, hora, estado,
id_empleado [FK → veterinario],
id_mascota [FK → mascota])

diagnostico (id_diagnostico [PK], fecha_diagnostico, descripcion,
id_cita [FK → cita])

tratamiento (id_tratamiento [PK], descripcion, estado,
id_diagnostico [FK → diagnostico])

medicamento (id_medicamento [PK], nombre_medicamento)

detalle_tratamiento (id_tratamiento [PK, FK → tratamiento],
id_medicamento [PK, FK → medicamento],
duracion_dias)

alergia (id_mascota [PK, FK → mascota],
id_medicamento [PK, FK → medicamento],
gravedad)

factura (id_factura [PK], total, metodo_pago,
id_cita [FK → cita])

3.3. Mapeo de relaciones a llaves foráneas

Relación conceptual	Cardinalidad	Representación en el modelo relacional
propietario – posee – mascota	1:N	id_propietario en mascota
especie – Pertenece – mascota	1:N	id_especie en mascota
mascota – agenda – cita	1:N	id_mascota en cita
veterinario – Atiende – cita	1:N	id_empleado en cita
cita – Genera – diagnostico	1:N	id_cita en diagnostico
cita – Emite – factura	1:1	id_cita en factura
diagnostico – Incluye – tratamiento	1:N	id_diagnostico en tratamiento
veterinario – especialidad	N:M	Tabla asociativa veterinario_especialidad
tratamiento – medicamento	N:M	Tabla asociativa detalle_tratamiento
mascota – medicamento (alergia)	N:M	Tabla asociativa alergia

3.4. Verificación de la Tercera Forma Normal

Para comprobar que el modelo está correctamente normalizado, se verificó que cumpliera con las tres primeras formas normales. En la **Primera Forma Normal (1FN)** se confirma que todos los atributos son atómicos, es decir, que cada campo guarda un único valor y que no existen grupos de datos que se repitan. En la **Segunda Forma Normal (2FN)** se revisa que cada atributo dependa de la llave primaria completa; en las tablas asociativas, atributos como fecha_obtencion, duracion_dias y gravedad dependen de la llave compuesta entera y no de una sola parte, por lo que no hay dependencias parciales. Finalmente, en la **Tercera Forma Normal (3FN)** se comprueba que no existan dependencias transitivas; por ejemplo, los datos de la especie se guardan únicamente en la tabla especie y se referencian desde mascota por medio de id_especie, evitando así que esa información se repita en varias tablas.

Nota sobre un atributo derivado: el campo total de la tabla factura se podría calcular a partir de los servicios de la cita. Se decidió guardarlo de forma directa para que las consultas de facturación sean más rápidas, lo cual es una desnormalización hecha a propósito. Esto no rompe la 3FN siempre que su valor se mantenga consistente mediante procedimientos o triggers.

4. Diccionario de datos

En esta sección se detalla cada una de las tablas de la base de datos junto con sus campos. Para cada campo se indica el tipo de dato, si es llave primaria o foránea, si admite valores nulos y una breve descripción de para qué sirve. Los tipos de dato se expresan en la sintaxis de PostgreSQL, y los identificadores principales se manejan como `BIGINT GENERATED ALWAYS AS IDENTITY`, que es un entero grande que se autoincrementa de forma automática.

veterinario

Campo	Tipo	Llave	Nulidad	Descripción
id_empleado	BIGINT	PK	NOT NULL	Identificador único del veterinario (autogenerado).
nombre_veterinario	VARCHAR(50)		NOT NULL	Nombre del veterinario.
salario	NUMERIC(8,2)		NOT NULL	Salario mensual asignado.

especialidad

Campo	Tipo	Llave	Nulidad	Descripción
id_especialidad	BIGINT	PK	NOT NULL	Identificador único de la especialidad (autogenerado).
nombre_especialidad	VARCHAR(100)		NOT NULL	Denominación de la especialidad clínica.

veterinario_especialidad

Campo	Tipo	Llave	Nulidad	Descripción
id_especialidad	BIGINT	PK, FK	NOT NULL	Especialidad obtenida. FK → especialidad.
id_empleado	BIGINT	PK, FK	NOT NULL	Veterinario que la obtiene. FK

				→ veterinario.
fecha_obtencion	DATE		NOT NULL	Fecha en que el veterinario obtuvo la especialidad.

ropietario

Campo	Tipo	Llave	Nulidad	Descripción
id_propietario	BIGINT	PK	NOT NULL	Identificador único del propietario (autogenerado).
nombre_propietario	VARCHAR(100)		NOT NULL	Nombre del propietario.
telefono	VARCHAR(20)		NOT NULL	Número de contacto telefónico.
fecha_registro	DATE		NOT NULL	Fecha de alta del

				propietario en el sistema.
--	--	--	--	----------------------------

especie

Campo	Tipo	Llave	Nulidad	Descripción
id_especie	BIGINT	PK	NOT NULL	Identificador único de la especie (autogenerado).
nombre_especie	VARCHAR(50)		NOT NULL	Nombre de la especie (canino, felino, etc.).
familia	VARCHAR(50)		NOT NULL	Familia taxonómica de la especie.

mascota

Campo	Tipo	Llave	Nulidad	Descripción
id_mascota	BIGINT	PK	NOT NULL	Identificador único de la mascota (autogenerado).
nombre_mascota	VARCHAR(50)		NOT NULL	Nombre de la mascota.
peso	NUMERIC(6, 2)		NOT NULL	Peso en kilogramos.
fecha_nacimiento	DATE		NOT NULL	Fecha de nacimiento de la mascota.
id_especie	BIGINT	FK	NOT NULL	Especie a la que pertenece. FK → especie.
id_propietario	BIGINT	FK	NOT NULL	Propietario de la mascota. FK → propietario.

cita

Campo	Tipo	Llave	Nulidad	Descripción
id_cita	BIGINT	PK	NOT NULL	Identificador único de la cita (autogenerado).
fecha	DATE		NOT NULL	Fecha de la cita.
hora	TIME		NOT NULL	Hora programada de la cita.
estado	VARCHAR(50)		NOT NULL	Estado de la cita. Valores permitidos: PERDIDA, PENDIENTE, ATENDIDA.
id_empleado	BIGINT	FK	NOT NULL	Veterinario

				que atiende. FK → veterinario.
id_mascota	BIGINT	FK	NOT NULL	Mascota atendida. FK → mascota.

diagnostico

Campo	Tipo	Llave	Nulidad	Descripción
id_diagnostico	BIGINT	PK	NOT NULL	Identificador único del diagnóstico (autogenerado).
fecha_diagnostico	DATE		NOT NULL	Fecha de emisión del diagnóstico.
descripcion	VARCHAR(150)		NOT NULL	Detalle clínico del diagnóstico.
id_cita	BIGINT	FK	NOT NULL	Cita que

				originó el diagnóstico. FK → cita.
--	--	--	--	---------------------------------------

tratamiento

Campo	Tipo	Llave	Nulidad	Descripción
id_tratamiento	BIGINT	PK	NOT NULL	Identificador único del tratamiento (autogenerado).
descripcion	VARCHAR(150)		NOT NULL	Descripción del tratamiento indicado.
estado	VARCHAR(150)		NOT NULL	Estado del tratamiento. Valores permitidos: ACTIVO, FINALIZADO

				O, SUSPENDID O.
id_diagnostico	BIGINT	FK	NOT NULL	Diagnóstico asociado. FK → diagnostico.

medicamento

Campo	Tipo	Llave	Nulidad	Descripción
id_medimento	BIGINT	PK	NOT NULL	Identificador único del medicamento (autogenerado).
nombre_medimento	VARCHAR(60)		NOT NULL	Nombre del medicamento.

detalle_tratamiento

Campo	Tipo	Llave	Nulidad	Descripción
id_tratamiento	BIGINT	PK, FK	NOT NULL	Tratamiento al que pertenece el detalle. FK → tratamiento.
id_medimento	BIGINT	PK, FK	NOT NULL	Medicamento prescrito. FK → medicamento.
duracion_dias	INT		NOT NULL	Duración en días de la administración del medicamento.

alergia

Campo	Tipo	Llave	Nulidad	Descripción
id_mascota	BIGINT	PK, FK	NOT NULL	Mascota que presenta la alergia. FK → mascota.
id_medicamente	BIGINT	PK, FK	NOT NULL	Medicamento que causa la alergia. FK → medicamento.
gravedad	VARCHAR(30)		NOT NULL	Nivel de gravedad. Valores permitidos: LEVE, MEDIA, ALTA.

factura

Campo	Tipo	Llave	Nulidad	Descripción
id_factura	BIGINT	PK	NOT NULL	Identificador único de la factura (autogenerado).
total	NUMERIC(6, 2)		NOT NULL	Monto total facturado.
metodo_pago	VARCHAR(50)		NOT NULL	Método de pago. Valores permitidos: TARJETA, TRANSACCION, EFECTIVO.
id_cita	BIGINT	FK	NOT NULL	Cita facturada. FK → cita (relación 1:1).

5. Documentación de consultas más frecuentes

A continuación, se documentan las consultas que más se utilizan dentro del sistema. Para cada una se explica brevemente qué hace y luego se muestra el código en SQL. Estas consultas responden a las necesidades de información planteadas en el escenario, como conocer el historial de una mascota o saber cuánto se ha facturado.

5.1. Historial médico completo de una mascota

Esta consulta permite obtener el historial médico completo de una mascota. Para lograrlo se unen las tablas de citas, diagnósticos, tratamientos y medicamentos, de manera que se pueda ver todo lo que se le ha hecho al paciente en un solo resultado. Se utilizan LEFT JOIN en la parte de los medicamentos para que también aparezcan los tratamientos que todavía no tienen un medicamento asignado.

```
SELECT
    m.nombre_mascota    AS mascota,
    c.fecha             AS fecha_cita,
    c.hora,
    v.nombre_veterinario AS veterinario,
    d.descripcion       AS diagnostico,
    t.descripcion       AS tratamiento,
    t.estado            AS estado_tratamiento,
    med.nombre_medimento AS medicamento,
    dt.duracion_dias
FROM mascota m
JOIN cita c      ON c.id_mascota = m.id_mascota
```

```

JOIN veterinario v          ON v.id_empleado = c.id_empleado
JOIN diagnostico d          ON d.id_cita      = c.id_cita
JOIN tratamiento t          ON t.id_diagnostico = d.id_diagnostico
LEFT JOIN detalle_tratamiento dt ON dt.id_tratamiento = t.id_tratamiento
LEFT JOIN medicamento med    ON med.id_medicamento = dt.id_medicamento
WHERE m.id_mascota = 1
ORDER BY c.fecha, c.hora;

```

5.2. Reporte de facturación mensual agregado por método de pago

Esta consulta sirve para generar un reporte de los ingresos de la clínica. Los datos se agrupan por año, mes y método de pago, y por cada grupo se muestra la cantidad de facturas y el total recaudado. De esta forma se puede saber cuánto entró en cada mes y qué método de pago se usa más.

```

SELECT
    EXTRACT(YEAR FROM c.fecha) AS anio,
    EXTRACT(MONTH FROM c.fecha) AS mes,
    f.metodo_pago,
    COUNT(*)                AS total_facturas,
    SUM(f.total)            AS ingreso_total
FROM factura f
JOIN cita c ON c.id_cita = f.id_cita
GROUP BY anio, mes, f.metodo_pago
ORDER BY anio, mes, f.metodo_pago;

```

5.3. Medicamentos prescritos con mayor frecuencia

Esta consulta muestra cuáles son los medicamentos que más se han recetado, contando cuántas veces aparece cada uno en los detalles de tratamiento.

```
SELECT
    med.nombre_medicamento,
    COUNT(*) AS veces_recetado
FROM detalle_tratamiento dt
JOIN medicamento med ON med.id_medicamento = dt.id_medicamento
GROUP BY med.id_medicamento, med.nombre_medicamento
ORDER BY veces_recetado DESC;
```

5.4. Veterinario con mayor número de citas en el trimestre actual

Esta consulta cuenta cuántas citas ha tenido cada veterinario durante el trimestre actual, para identificar al que más pacientes ha atendido en ese periodo.

```
SELECT
    v.nombre_veterinario,
    COUNT(*) AS num_citas
FROM cita c
JOIN veterinario v ON v.id_empleado = c.id_empleado
WHERE c.fecha >= DATE_TRUNC('quarter', CURRENT_DATE)
    AND c.fecha < DATE_TRUNC('quarter', CURRENT_DATE) + INTERVAL '3 months'
GROUP BY v.id_empleado, v.nombre_veterinario
ORDER BY num_citas DESC;
```

5.5. Propietarios con mascotas sin citas en los últimos 6 meses

Esta consulta busca a los propietarios cuyas mascotas no han asistido a ninguna cita en los últimos seis meses, lo que sirve para darles seguimiento o recordarles que vuelvan a la clínica.

```
SELECT DISTINCT
    p.id_propietario,
    p.nombre_propietario,
    p.telefono
FROM propietario p
JOIN mascota m ON m.id_propietario = p.id_propietario
WHERE NOT EXISTS (
    SELECT 1
    FROM cita c
    WHERE c.id_mascota = m.id_mascota
    AND c.fecha >= CURRENT_DATE - INTERVAL '6 months'
);
```

6. Funciones, procedimientos almacenados y disparadores

En esta última sección se documenta la lógica programable de la base de datos. Aquí se incluyen una función, un procedimiento almacenado y un disparador, los cuales sirven para automatizar tareas y para mantener la integridad de los datos. Para cada uno se explica qué hace, se muestra su código en PostgreSQL y se incluye un ejemplo de cómo se utiliza.

6.1. Función: fn_historial_clinico

Esta función devuelve el historial clínico completo de una mascota a partir de su identificador. En lugar de tener que escribir la consulta completa cada vez, basta con llamar a la función pasándole el id de la mascota y esta retorna todas las filas con sus citas, diagnósticos, tratamientos y medicamentos. Con esto se cumple el procedimiento que pedía el escenario.

```
CREATE OR REPLACE FUNCTION fn_historial_clinico(p_id_mascota BIGINT)
RETURNS TABLE (
    fecha_cita    DATE,
    veterinario   TEXT,
    diagnostico   TEXT,
    tratamiento   TEXT,
    medicamento   TEXT,
    duracion_dias INT
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
```



```

SELECT

    c.fecha,

    v.nombre_veterinario::TEXT,

    d.descripcion::TEXT,

    t.descripcion::TEXT,

    med.nombre_medicamento::TEXT,

    dt.duracion_dias

FROM cita c

JOIN veterinario v      ON v.id_empleado  = c.id_empleado

JOIN diagnostico d      ON d.id_cita      = c.id_cita

JOIN tratamiento t      ON t.id_diagnostico = d.id_diagnostico

LEFT JOIN detalle_tratamiento dt ON dt.id_tratamiento = t.id_tratamiento

LEFT JOIN medicamento med  ON med.id_medicamento = dt.id_medicamento

WHERE c.id_mascota = p_id_mascota

ORDER BY c.fecha;

END;

$$;

```

-- Invocación:

```
SELECT * FROM fn_historial_clinico(1);
```

6.2. Procedimiento almacenado: sp_RegistrarCita

Este procedimiento se encarga de registrar una nueva cita, pero antes de insertarla revisa que el veterinario no tenga ya otra cita en la misma fecha y hora. Si encuentra un choque de horario, no

permite guardar la cita y muestra un mensaje de error. De esta manera se cumple la regla RN-11 y se evita que un mismo veterinario quede agendado dos veces al mismo tiempo.

```
CREATE OR REPLACE PROCEDURE sp_RegistrarCita(
    p_fecha    DATE,
    p_hora     TIME,
    p_estado   VARCHAR,
    p_id_empleado BIGINT,
    p_id_mascota BIGINT
)
LANGUAGE plpgsql
AS $$
BEGIN
    IF EXISTS (
        SELECT 1
        FROM cita
        WHERE id_empleado = p_id_empleado
            AND fecha = p_fecha
            AND hora = p_hora
            AND estado <> 'PERDIDA'
    ) THEN
        RAISE EXCEPTION 'El veterinario % ya tiene una cita activa el % a las %.',
            p_id_empleado, p_fecha, p_hora;
    END IF;

    INSERT INTO cita (fecha, hora, estado, id_empleado, id_mascota)
```

```
VALUES (p_fecha, p_hora, p_estado, p_id_empleado, p_id_mascota);
END;
$$;
```

```
-- Invocación:
```

```
CALL sp_RegistrarCita('2026-06-20', '10:00', 'PENDIENTE', 1, 3);
```

6.3. Disparador (trigger): tr_ValidarAlergiasAntesTratamiento

Este disparador funciona como una medida de seguridad para los pacientes. Cada vez que se intenta agregar un medicamento a un tratamiento (es decir, insertar una fila en detalle_tratamiento), el trigger revisa primero si la mascota tiene una alergia registrada a ese medicamento. Para saber de qué mascota se trata, navega desde el tratamiento hacia el diagnóstico y luego hacia la cita. Si encuentra una alergia, bloquea la inserción y muestra un error, cumpliendo así la regla RN-12. El disparador se ejecuta BEFORE INSERT, o sea, antes de que el dato se guarde.

```
CREATE OR REPLACE FUNCTION fn_validar_alergias()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_id_mascota BIGINT;
BEGIN
    -- Localizar la mascota asociada al tratamiento en cuestión
```

```

SELECT c.id_mascota
      INTO v_id_mascota
FROM tratamiento t
JOIN diagnostico d ON d.id_diagnostico = t.id_diagnostico
JOIN cita c      ON c.id_cita      = d.id_cita
WHERE t.id_tratamiento = NEW.id_tratamiento;

-- Verificar la existencia de una alergia registrada
IF EXISTS (
      SELECT 1
      FROM alergia
      WHERE id_mascota  = v_id_mascota
            AND id_medimento = NEW.id_medimento
) THEN
      RAISE EXCEPTION
            'Operación bloqueada: la mascota % presenta una alergia registrada al medicamento %.',
            v_id_mascota, NEW.id_medimento;
END IF;

RETURN NEW;

END;

$$;

CREATE TRIGGER tr_ValidarAlergiasAntesTratamiento
BEFORE INSERT ON detalle_tratamiento

```

```
FOR EACH ROW
```

```
EXECUTE FUNCTION fn_validar_alergias();
```

Comportamiento esperado: si se intenta registrar un medicamento al cual la mascota es alérgica, la operación se cancela y PostgreSQL devuelve el mensaje de error definido en la función. De esta forma el sistema protege al paciente y no permite guardar una prescripción que podría ser peligrosa.